VCF 9.0.1 Command Reference

Table of Contents

1. SDDC Manager API
2. SDDC Manager Services & Logs
3. SDDC Manager PostgreSQL Database
4. SDDC Manager SSH & Account Recovery
5. NSX Manager API
6. NSX Admin CLI
7. NSX Node Services
8. vCenter Server Management
9. vCenter / vSphere API
10. ESXi Host — System
11. ESXi Host — Network
12. ESXi Host — Storage
13. ESXi Host — VM Management
14. ESXi Host — Services & Troubleshooting
15. vSAN Operations
16. vSAN Disk Management
17. vmkfstools — Disk Cloning & Management
18. Certificate Operations — OpenSSL
19. Certificate Operations — Keytool
20. NSX Certificate Replacement (Full Workflow)
21. VCF Operations for Logs — Certificate
22. Offline Depot — Server & Bundles
23. SOS Diagnostic Bundle
24. VDT (VMware Depot Tool)
25. ovftool — Manual OVA Deployment
26. Windows Host — DNS & Firewall
27. Diagnostic Python Scripts
28. GRUB Recovery — SDDC Manager

---

1. SDDC Manager API

Get Auth Token

```
TOKEN=$(curl -sk -X POST https://sddc-manager.lab.local/v1/tokens \
  -H "Content-Type: application/json" \
  -d '{"username":"administrator@vsphere.local","password":"<pass>"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['accessToken'])")
```

Core Endpoints

```
# System health
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/system/health

# NSX cluster status (check for ACTIVE vs ACTIVATING)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/nsxt-clusters |
python3 -m json.tool

# Resource locks (stale locks block all operations)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/resource-locks |
python3 -m json.tool

# Credentials
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/credentials |
python3 -m json.tool

# Task details (UI hides error payloads)
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/tasks/<task-id> |
python3 -m json.tool

# System notifications
curl -sk -H "Authorization: Bearer $TOKEN"
https://localhost/v1/system/notifications

# Domains
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/domains | python3
-m json.tool

# Hosts
curl -sk -H "Authorization: Bearer $TOKEN" https://localhost/v1/hosts | python3 -
m json.tool

# Failed tasks (newest first)
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost/v1/tasks?
status=FAILED&pageSize=5&sortOrder=DESC" | python3 -m json.tool
```

```
# Stuck IN_PROGRESS tasks (these block new operations)
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost/v1/tasks?
status=IN_PROGRESS" | python3 -c \
  "import sys,json; d=json.load(sys.stdin); print(f'Count:
{len(d.get(\"elements\",[]))}'); [print(f'  {t[\"type\"]} |
{t[\"creationTimestamp\"]}') for t in d.get('elements',[])]"

# NSX credentials
curl -sk -H "Authorization: Bearer $TOKEN" "https://localhost/v1/credentials?
resourceType=NSXT_MANAGER" | python3 -m json.tool

# Cancel stuck task (NOTE: API often rejects with TA_TASK_CAN_NOT_BE_RETRIED — use
DB fix instead)
curl -sk -X PATCH https://localhost/v1/tasks/<task-id> \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"status":"CANCELLED"}'
```

---

2. SDDC Manager Services & Logs

Service Management

```
# Check all VCF services
systemctl status domainmanager lcm operationsmanager commonsvcs postgresql nginx

# Restart all services (recommended method)
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh

# Restart individual services
systemctl restart domainmanager
systemctl restart lcm
systemctl restart operationsmanager
systemctl restart commonsvcs
systemctl restart sddc-manager-ui-app.service

# List all VCF service units
systemctl list-units --type=service | grep vcf
```

Key Log Locations

```
# Domain manager (deployment, bringup)
tail -100 /var/log/vmware/vcf/domainmanager/domainmanager.log

# Operations manager (credential rotation, tasks)
tail -100 /var/log/vmware/vcf/operationsmanager/operationsmanager.log

# Lifecycle manager (bundles, upgrades, certs)
tail -100 /var/log/vmware/vcf/lcm/lcm.log
tail -100 /var/log/vmware/vcf/lcm/lcm-debug.log

# Common services
tail -100 /var/log/vmware/vcf/commonsvcs/commonsvcs.log
```

Log Search

```
# Search for errors
grep -ri "error\|exception\|failed"
/var/log/vmware/vcf/domainmanager/domainmanager.log | tail -50

# Search for cert/TLS issues
grep -i "tlsfatal\|ssl\|certificate" /var/log/vmware/vcf/lcm/lcm-debug.log | tail
-20

# Search by timestamp
grep "2026-02-22 14:" /var/log/vmware/vcf/operationsmanager/operationsmanager.log

# Follow log in real time
tail -f /var/log/vmware/vcf/domainmanager/domainmanager.log

# Search by task ID
grep "<task-id>" /var/log/vmware/vcf/lcm/lcm.log

# Journalctl
journalctl -u vcf-services --since "1 hour ago" -p err
```

Service Properties Ownership Fix

```
chown vcf:vcf /opt/vmware/vcf/domainmanager/conf/application-prod.properties
chown vcf:vcf /opt/vmware/vcf/operationsmanager/conf/application-prod.properties
```

NFS Mount Ownership Fix

```
chown root:vcf /nfs/vmware/vcf/nfs-mount/
```

---

3. SDDC Manager PostgreSQL Database

Enable Database Access (Trust Auth Workaround)

```
# Backup pg_hba.conf
cp /data/pgdata/pg_hba.conf /data/pgdata/pg_hba.conf.bak

# Set trust auth (temporary — ALWAYS restore after)
sed -i 's/scram-sha-256/trust/g' /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"
```

Connect to Databases

```
# IMPORTANT: Always disable psql pager to prevent --More-- prompts
export PAGER=cat
export PGPAGER=cat

# Platform database (resource states, locks, task metadata)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform"

# Operations manager database (tasks, executions)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d operationsmanager"

# List all databases
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -t -c \"SELECT datname FROM
pg_database WHERE datistemplate = false;\""
```

```
# Returns: postgres, lcm, platform, operationsmanager, domainmanager,
sddc_manager_ui

# List tables in a database
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -t -c \"SELECT tablename
FROM pg_tables WHERE schemaname = 'public' ORDER BY tablename;\""
```

Credential Cascade Failure — Full Repair

```
# ── Step 1: Fix NSX resource status (platform DB) ──
# Check current status (should be ACTIVE, may be ACTIVATING or ERROR)
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -t -c \"SELECT id,
status FROM nsxt;\""

# Fix ANY non-ACTIVE status
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE nsxt SET
status = 'ACTIVE' WHERE status != 'ACTIVE';\""

# ── Step 2: Clear stale resource locks (platform DB) ──
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT count(*)
FROM lock;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
lock;\""

# ── Step 3: Mark stuck tasks as resolved (platform DB) ──
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"SELECT resolved,
count(*) FROM task_metadata GROUP BY resolved;\""
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"UPDATE
task_metadata SET resolved = true WHERE resolved = false;\""

# ── Step 4: Clear task locks (platform DB) ──
su - postgres -c "PAGER=cat psql -h 127.0.0.1 -d platform -c \"DELETE FROM
task_lock;\""
```

Key tables: `platform` *DB:* `nsxt` *(status),* `lock` *,* `task_metadata` *(resolved),* `task_lock` *.* `operationsmanager` *DB:* `task` *(column:* `state` *, not* `status` *),* `execution` *(column:* `execution_status` *),* `processing_task` *,* `execution_to_task` *.*

Restore pg_hba.conf (CRITICAL — Do Immediately)

```
cp /data/pgdata/pg_hba.conf.bak /data/pgdata/pg_hba.conf
su - postgres -c "/usr/pgsql/15/bin/pg_ctl reload -D /data/pgdata"

# Verify restored (should return 4+)
grep -c 'scram-sha-256' /data/pgdata/pg_hba.conf
```

Restart After DB Fix

```
systemctl restart operationsmanager
systemctl is-active operationsmanager
```

---

4. SDDC Manager SSH & Account Recovery

SSH Access

```
# Only vcf user can SSH (root/admin rejected)
ssh vcf@192.168.1.241

# Escalate to root
su -

# File transfer (SCP doesn't work — restricted shell)
ssh vcf@192.168.1.241 "cat > /home/vcf/myfile.zip" < localfile.zip
ssh vcf@192.168.1.241 "cat /path/to/remote/file" > local_copy
```

Account Lockout (faillock)

```
# Check lockout status
faillock --user vcf

# Reset locked accounts (run from console as root)
```

```
faillock --user vcf --reset
faillock --user root --reset
```

---

5. NSX Manager API

Cluster & Node Status

```
# Cluster health (via VIP)
curl -sk -u admin:'<pass>' https://192.168.1.70/api/v1/cluster/status

# Cluster health (via node — use when VIP is down)
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/cluster/status

# Cluster info + node UUIDs
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/cluster
```

Transport & Networking

```
# Transport node status
curl -sk -u admin:'<pass>' https://192.168.1.70/api/v1/transport-nodes/state

# Transport zones
curl -sk -u admin:'<pass>' https://192.168.1.70/api/v1/transport-zones

# Segments
curl -sk -u admin:'<pass>' https://192.168.1.70/policy/api/v1/infra/segments

# Compute managers
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/fabric/compute-managers
```

Service Health

```
# Individual service status
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/node/services/http/status
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/node/services/search/status
```

```
# Node system status (includes load average)
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/node/status
```

Certificate Management

```
# List all certificates
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/trust-
management/certificates

# Import certificate (see Section 20 for full workflow)
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates?action=import" \
  -H "Content-Type: application/json" -d @/tmp/nsx-import.json
```

---

6. NSX Admin CLI

SSH as `admin` user (not root):

```
get cluster status                    # Cluster health
get managers                          # List manager nodes
get name-servers                      # DNS config
set name-servers <ip>                 # Set DNS (NOT via UI)
get ntp-servers                       # NTP config
set ntp-servers <ip>                  # Set NTP (NOT via UI)
```

---

7. NSX Node Services

SSH as `root` user:

```
# Service status
systemctl status proton               # Main NSX Manager API
systemctl status search               # Search service
systemctl status nsx-appl-proxy       # Appliance proxy
systemctl status nsx-sha              # System health agent
```

```
systemctl status corfu_server        # Distributed datastore

# Service restart
systemctl restart proton
systemctl restart search
systemctl restart nsx-appl-proxy

# Load monitoring (nested: load >30 = still booting)
cat /proc/loadavg
ps -eo pid,%cpu,%mem,comm --sort=-%cpu | head -15

# Disk space
df -h
```

---

8. vCenter Server Management

Service Control

```
# Status
service-control --status              # All services summary
service-control --status --all        # All services detailed
service-control --status vpxd         # Specific service

# Start / Stop / Restart
service-control --start --all
service-control --stop vpxd
service-control --restart vpxd
service-control --restart --all

# Alternative service listing
vmon-cli --list
vmon-cli --status vpxd
```

Certificate Manager

```
/usr/lib/vmware-vmca/bin/certificate-manager
```

```
# List certificate stores
for store in MACHINE_SSL_CERT TRUSTED_ROOTS; do
  /usr/lib/vmware-vmafd/bin/vecs-cli entry list --store $store
done
```

Database

```
# Check PostgreSQL
service-control --status vmware-vpostgres

# Connect
/opt/vmware/vpostgres/current/bin/psql -U postgres

# Active connections
/opt/vmware/vpostgres/current/bin/psql -U postgres -c "SELECT count(*) FROM
pg_stat_activity;"
```

Log Locations

```
# VPXD (main vCenter service)
tail -100 /var/log/vmware/vpxd/vpxd.log
grep -i "error\|exception" /var/log/vmware/vpxd/vpxd.log | tail -50

# UI
tail -100 /var/log/vmware/vsphere-ui/logs/vsphere_client_virgo.log

# First boot (during deployment)
cat /var/log/firstboot/firstbootStatus.json
tail -50 /var/log/vmware/firstboot/installer.log
```

Appliance Info

```
vamicli version --appliance
vamicli update --check
```

---

9. vCenter / vSphere API

```
# Create session
SESSION=$(curl -sk -X POST https://vcenter.lab.local/api/session \
  -u 'administrator@vsphere.local:<pass>' | tr -d '"')

# List VMs
curl -sk -H "vmware-api-session-id: $SESSION"
https://vcenter.lab.local/api/vcenter/vm

# MOB (browser-based deep object access)
# https://vcenter.lab.local/mob
```

---

10. ESXi Host — System

```
esxcli system hostname get                              # Show FQDN
esxcli system hostname set --fqdn=esxi01.lab.local      # Set FQDN
esxcli system version get                                # ESXi version/build
esxcli system time get                                   # System time
esxcli system maintenanceMode get                        # Check maintenance mode
esxcli system maintenanceMode set -e true -m noAction    # Enter (no data
evacuation)
esxcli system maintenanceMode set -e true -m evacuateAllData  # Enter (vSAN
evacuate)
esxcli system maintenanceMode set -e false               # Exit maintenance mode
esxcli system ssh set --enable=true                      # Enable SSH
esxcli system ssh get                                    # SSH status
esxcli hardware cpu global get                           # CPU info (EVC check)
```

---

11. ESXi Host — Network

```
# VMkernel adapters
esxcli network ip interface list
```

```
esxcli network ip interface ipv4 get -i vmk0
esxcli network ip interface add -i vmk1 -p "vMotion"
esxcli network ip interface remove -i vmk1

# Routing
esxcli network ip route ipv4 list
esxcli network ip neighbor list

# vSwitch
esxcli network vswitch standard list
esxcli network vswitch dvs vmware list

# Physical NICs
esxcli network nic list
esxcli network nic get -n vmnic0
esxcli network nic stats get -n vmnic0

# Connections / Firewall
esxcli network ip connection list
esxcli network ip connection list | grep 1234
esxcli network firewall ruleset list
esxcli network firewall ruleset list | grep -i ssh

# vmkping
vmkping 192.168.1.75                          # Basic ping
vmkping -I vmk2 192.168.1.75                  # Ping from specific vmk
vmkping -d -s 1572 192.168.1.75               # Jumbo frame test

# esxcfg
esxcfg-vmknic -l                              # VMkernel NIC list
esxcfg-vswitch -l                             # vSwitch list
esxcfg-nics -l                                # Physical NIC list
```

---

12. ESXi Host — Storage

```
# Device listing
esxcli storage core device list
esxcli storage core device list | grep -E "Display Name|Is SSD"
esxcli storage core adapter list
esxcli storage filesystem list
```

```
esxcli storage vmfs extent list

# Rescan
esxcli storage core adapter rescan --all

# SSD detection (force for nested)
esxcli storage nmp satp rule add -s VMW_SATP_LOCAL -d <device> -o enable_ssd
esxcli storage core claiming reclaim -d <device>
esxcli storage nmp satp rule list | grep enable_ssd

# Disk eligibility for vSAN
vdq -qH
vdq -q -d <device>

# Partition management
partedUtil getptbl /vmfs/devices/disks/<device>
partedUtil delete /vmfs/devices/disks/<device> 1
partedUtil delete /vmfs/devices/disks/<device> 2
```

---

13. ESXi Host — VM Management

```
# List VMs
vim-cmd vmsvc/getallvms

# Power operations
vim-cmd vmsvc/power.getstate <vmid>
vim-cmd vmsvc/power.on <vmid>
vim-cmd vmsvc/power.off <vmid>
vim-cmd vmsvc/power.shutdown <vmid>
vim-cmd vmsvc/power.reset <vmid>

# Register / Unregister
vim-cmd solo/registervm "/vmfs/volumes/<datastore>/<vm>/<vm>.vmx"
vim-cmd vmsvc/unregister <vmid>

# Device info
vim-cmd vmsvc/device.getdevices <vmid>

# BIOS boot
vim-cmd vmsvc/setboot.options <vmid> enterBIOSSetup=true
```

```
# Hardware info
vim-cmd hostsvc/hosthardware

# Maintenance mode
vim-cmd hostsvc/maintenance_mode_enter
vim-cmd hostsvc/maintenance_mode_exit
```

---

14. ESXi Host — Services & Troubleshooting

Service Management

```
# Host services
/etc/init.d/hostd restart
/etc/init.d/hostd status
/etc/init.d/vpxa restart
/etc/init.d/vpxa status
services.sh restart                          # Restart all host services

# SSH
/etc/init.d/SSH start
/etc/init.d/SSH stop
/etc/init.d/SSH status
vim-cmd hostsvc/enable_ssh && vim-cmd hostsvc/start_ssh

# NSX services on host
/etc/init.d/nsx-proxy status
/etc/init.d/nsx-proxy restart
/etc/init.d/nsx-datapath status
/etc/init.d/nsx-opsagent status

# Auto-backup
/sbin/auto-backup.sh
```

Certificate Regeneration

```
mv /etc/vmware/ssl/rui.crt /etc/vmware/ssl/rui.crt.bak
mv /etc/vmware/ssl/rui.key /etc/vmware/ssl/rui.key.bak
/sbin/generate-certificates
services.sh restart
openssl x509 -in /etc/vmware/ssl/rui.crt -text -noout | grep -A1 "Subject
Alternative Name"
```

Get Host Thumbprints

```
echo | openssl s_client -connect 192.168.1.74:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.75:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.76:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
echo | openssl s_client -connect 192.168.1.82:443 2>/dev/null | openssl x509 -
noout -fingerprint -sha256
```

Performance / Diagnostics

```
esxtop                                        # Interactive performance monitor
esxtop -b -d 5 -n 10 > /tmp/esxtop.csv       # Batch mode to CSV
```

vMotion Fix (vhv.enable Ghost Setting)

```
# Diagnose
grep -i vhv /vmfs/volumes/<datastore>/<vm>/<vm>.vmx

# Fix (removing the line is NOT enough — must set explicitly)
echo 'vhv.enable = "FALSE"' >> /vmfs/volumes/<datastore>/<vm>/<vm>.vmx
```

NSX Host Logs

```
tail -50 /var/log/nsx-syslog.log
esxcli network ip connection list | grep 1234
esxcfg-vmknic -l | grep -i tep
```

---

15. vSAN Operations

Health & Status

```
esxcli vsan health cluster list              # Health summary
esxcli vsan health cluster get               # Detailed health
esxcli vsan health cluster get -t "Network health"
esxcli vsan cluster get                      # Cluster info
esxcli vsan network list                     # vSAN network config
esxcli vsan debug resync summary get         # Resync status
esxcli vsan debug object health summary get  # Object health
esxcli vsan debug object list                # All objects
```

Storage & Disk Groups

```
esxcli vsan storage list                     # Disk/storage status
esxcli vsan storage automode set --enabled=false
esxcli vsan storage automode set --enabled=true
esxcli vsan storage add -s <cache> -d <capacity>
esxcli vsan storage remove -s <device>
```

Network

```
esxcli vsan network ip add -i vmk1
esxcli vsan network ip remove -i vmk1
vmkping -I vmk2 192.168.12.120               # TEP-to-TEP connectivity
```

Logging

```
grep -i "vsan\|cmmds\|clom\|dom\|lsom" /var/log/vmkernel.log | tail -50
tail -50 /var/log/vsanmgmt.log
tail -50 /var/log/vsantraced.log
/etc/init.d/vsanmgmtd status
/etc/init.d/vsand status
```

vSAN Observer (RVC)

```
rvc administrator@vsphere.local@localhost
cd /vcenter.lab.local/vcenter-dc01/computers/vcenter-cl01
vsan.observer . --run-webserver --force
```

---

16. vSAN Disk Management

Remove Disk from vSAN & Clean Partitions

```
esxcli vsan storage remove -d <device>
partedUtil getptbl /vmfs/devices/disks/<device>
partedUtil delete /vmfs/devices/disks/<device> 1
partedUtil delete /vmfs/devices/disks/<device> 2
vdq -q                                      # Verify eligible again
```

---

17. vmkfstools — Disk Cloning & Management

```
# Clone thick to thin
vmkfstools -i <source>.vmdk <dest>.vmdk -d thin

# Clone thick to thick
vmkfstools -i <source>.vmdk <dest>.vmdk -d zeroedthick
vmkfstools -i <source>.vmdk <dest>.vmdk -d eagerzeroedthick
```

```
# Create new disk
vmkfstools -c 50G -d thin /vmfs/volumes/<ds>/<vm>/newdisk.vmdk

# Extend disk
vmkfstools -X 100G /vmfs/volumes/<ds>/<vm>/disk.vmdk

# Delete disk
vmkfstools -U /vmfs/volumes/<ds>/<vm>/disk.vmdk

# Get disk geometry
vmkfstools -g /vmfs/volumes/<ds>/<vm>/disk.vmdk

# Disk descriptor info
vmkfstools -D "/vmfs/volumes/<ds>/<vm>/<vm>.vmdk"
```

---

18. Certificate Operations — OpenSSL

Generate Self-Signed Certificate

```
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt \
  -days 365 -nodes -subj "/CN=hostname"
```

Generate with SANs

```
openssl req -x509 -newkey rsa:2048 -keyout server.key -out server.crt \
  -days 365 -nodes -subj "/CN=192.168.1.52/O=VCF-Depot/C=US" \
  -addext "subjectAltName=IP:192.168.1.52,DNS:localhost" \
  -addext "keyUsage=digitalSignature,keyEncipherment" \
  -addext "extendedKeyUsage=serverAuth"
```

Generate CSR

```
openssl genrsa -out server.key 2048
```

```
openssl req -new -key server.key -out server.csr -subj "/CN=hostname/O=Org/C=US"
```

View Certificate Details

```
openssl x509 -in cert.crt -text -noout                              # Full details
openssl x509 -in cert.crt -noout -subject                           # Subject only
openssl x509 -in cert.crt -noout -issuer                            # Issuer only
openssl x509 -in cert.crt -noout -dates                             # Valid dates
openssl x509 -in cert.crt -noout -enddate                           # Expiry only
openssl x509 -in cert.crt -text -noout | grep -A2 "Subject Alternative"  # SANs
openssl x509 -in cert.crt -noout -fingerprint -sha256               # Fingerprint
```

Check Remote Certificate

```
# View remote cert
echo | openssl s_client -connect <host>:443 2>/dev/null | openssl x509 -noout -
text | grep -A2 "Subject Alternative"

# Download remote cert to file
openssl s_client -showcerts -connect <host>:443 < /dev/null 2>/dev/null | openssl
x509 -outform PEM > /tmp/cert.pem

# Test TLS connectivity
openssl s_client -connect <host>:<port>
openssl s_client -connect <host>:<port> -tls1_2
openssl s_client -connect <host>:<port> -tls1_2 </dev/null 2>&1 | grep -E
"Cipher|Protocol|Verify"
```

Convert Formats

```
openssl x509 -in cert.pem -outform der -out cert.der               # PEM → DER
openssl x509 -in cert.der -inform der -outform pem -out cert.pem   # DER → PEM
```

Verify Certificate

```
openssl verify -CAfile ca.crt server.crt
```

---

19. Certificate Operations — Keytool

List Certificates

```
keytool -list -keystore /etc/alternatives/jre/lib/security/cacerts -storepass
changeit
keytool -list -alias <alias> -keystore /etc/alternatives/jre/lib/security/cacerts
-storepass changeit -v
```

Import Certificate

```
# Into Java cacerts
keytool -importcert -alias <alias> -file /tmp/cert.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt

# Into VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias <alias> -file /tmp/cert.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt
```

Delete Certificate

```
keytool -delete -alias <alias> \
  -keystore /etc/alternatives/jre/lib/security/cacerts -storepass changeit
```

Find All Cacerts on System

```
find / -name "cacerts" -type f 2>/dev/null
```

Restart After Keystore Changes

```
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

---

20. NSX Certificate Replacement (Full Workflow)

Step 1 — Create OpenSSL Config

```
cat > /tmp/nsx-cert.conf << 'EOF'
[ req ]
default_bits = 2048
distinguished_name = req_distinguished_name
req_extensions = req_ext
x509_extensions = req_ext
prompt = no

[ req_distinguished_name ]
countryName = US
stateOrProvinceName = Lab
localityName = Lab
organizationName = lab.local
commonName = nsx-vip.lab.local

[ req_ext ]
basicConstraints = CA:FALSE
subjectAltName = @alt_names

[alt_names]
DNS.1 = nsx-vip.lab.local
DNS.2 = nsx-node1.lab.local
DNS.3 = nsx-manager.lab.local
IP.1 = 192.168.1.70
```

```
IP.2 = 192.168.1.71
EOF
```

Step 2 — Generate Certificate

```
openssl req -x509 -nodes -days 825 -newkey rsa:2048 \
  -keyout /tmp/nsx.key -out /tmp/nsx.crt \
  -config /tmp/nsx-cert.conf -sha256
```

Step 3 — Verify SANs

```
openssl x509 -in /tmp/nsx.crt -text -noout | grep -A4 "Subject Alternative Name"
```

Step 4 — Build JSON Payload

```
python -c "
import json
cert = open('/tmp/nsx.crt').read()
key = open('/tmp/nsx.key').read()
print(json.dumps({'pem_encoded': cert, 'private_key': key}))
" > /tmp/nsx-import.json
```

Step 5 — Import via NSX API

```
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates?action=import" \
  -H "Content-Type: application/json" -d @/tmp/nsx-import.json
```

Step 6 — Get Node UUID

```
curl -sk -u admin:'<pass>' https://192.168.1.71/api/v1/cluster
```

Step 7 — Apply to Node

```
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates/<cert-id>?
action=apply_certificate&service_type=API&node_id=<node-uuid>"
```

Step 8 — Apply to VIP

```
curl -sk -u admin:'<pass>' -X POST \
  "https://192.168.1.71/api/v1/trust-management/certificates/<cert-id>?
action=apply_certificate&service_type=MGMT_CLUSTER"
```

Step 9 — Verify

```
openssl s_client -connect 192.168.1.71:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
openssl s_client -connect 192.168.1.70:443 -showcerts </dev/null 2>/dev/null |
openssl x509 -noout -text | grep -A2 "Subject Alternative Name"
```

Step 10 — Import into SDDC Manager Trust Stores

```
# Pull active NSX cert
openssl s_client -showcerts -connect 192.168.1.71:443 < /dev/null 2>/dev/null |
openssl x509 -outform PEM > /tmp/nsx-root.crt

# VCF trust store
KEY=$(cat /etc/vmware/vcf/commonsvcs/trusted_certificates.key)
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/vmware/vcf/commonsvcs/trusted_certificates.store \
  -storepass "$KEY" -noprompt

# Java cacerts
keytool -importcert -alias nsx-selfsigned -file /tmp/nsx-root.crt \
  -keystore /etc/alternatives/jre/lib/security/cacerts \
  -storepass changeit -noprompt
```

```
# Restart services
/opt/vmware/vcf/operationsmanager/scripts/cli/sddcmanager_restart_services.sh
```

---

21. VCF Operations for Logs — Certificate

Create OpenSSL Config

```
cat > /tmp/vrli-cert.cnf << 'EOF'
[req]
default_bits = 4096
prompt = no
default_md = sha256
distinguished_name = dn
req_extensions = v3_req
x509_extensions = v3_req

[dn]
C = US
ST = California
L = Lab
O = Lab
OU = VCF
CN = logs.lab.local

[v3_req]
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth, clientAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = logs.lab.local
DNS.2 = logs
IP.1 = 192.168.1.242
EOF
```

Generate and Verify

```
openssl req -x509 -nodes -days 730 -newkey rsa:4096 \
  -keyout /tmp/vrli.key -out /tmp/vrli.crt -config /tmp/vrli-cert.cnf

openssl x509 -in /tmp/vrli.crt -noout -text | grep -A5 "Subject Alternative Name"
cat /tmp/vrli.crt
cat /tmp/vrli.key
```

---

22. Offline Depot — Server & Bundles

Windows — Start HTTPS Depot Server

```
cd C:\VCF-DEPOT
python https_server.py
```

Windows — Generate Certificate

```
cd C:\VCF-DEPOT
python generate_cert.py
```

Windows — Firewall Rule

```
netsh advfirewall firewall add rule name="VCF Depot 8443" dir=in action=allow
protocol=tcp localport=8443
```

Windows — Extract Metadata

```
Expand-Archive -Path "vcf-9.0.1.0-offline-depot-metadata.zip" -DestinationPath
"C:\VCF-Depot\metadata-extract" -Force
Copy-Item "C:\VCF-Depot\metadata-extract\PROD\*" "C:\VCF-Depot\PROD\" -Recurse -
Force
```

SDDC Manager — Import Depot Certificate

```
# Download cert from depot server
openssl s_client -connect 192.168.1.160:8443 </dev/null 2>/dev/null | openssl x509
-outform PEM > /tmp/depot.crt

# Import into Java truststore
CACERTS=$(find /usr -name cacerts 2>/dev/null | head -1)
keytool -importcert -trustcacerts -alias offline-depot -file /tmp/depot.crt \
  -keystore $CACERTS -storepass changeit -noprompt

# Verify
keytool -list -alias offline-depot -keystore $CACERTS -storepass changeit

# Restart LCM
systemctl restart lcm
```

SDDC Manager — Verify Depot Connectivity

```
curl -k -u admin:admin
https://192.168.1.160:8443/PROD/COMP/SDDC_MANAGER_VCF/lcm/productVersionCatalog/prod
```

---

23. SOS Diagnostic Bundle

```
cd /opt/vmware/sddc-support/

# Collect log bundle
./sos --domain-name mgmt --log-bundle

# Log bundle with health check
./sos --domain-name mgmt --log-bundle --health-check

# Health check only
./sos --health-check

# Get inventory
```

```
./sos --get-inventory

# Get all passwords
./sos --get-passwords

# Backup config
./sos --backup-config

# Cleanup old logs
./sos --cleanup-logs
```

---

24. VDT (VMware Depot Tool)

```
# Upload to SDDC Manager
ssh vcf@192.168.1.241 "cat > /home/vcf/vdt.zip" < vdt-2.2.7_02-05-2026.zip

# Extract and run
su -
cd /home/vcf
unzip vdt-2.2.7_02-05-2026.zip
cd vdt-2.2.7_02-05-2026
python vdt.py
# Prompted for administrator@vsphere.local password
```

> ***Note:*** *VDT is NOT pre-installed. Download from Broadcom KB 344917.*

---

25. ovftool — Manual OVA Deployment

Discover vCenter Inventory Path

```
ovftool --noSSLVerify "vi://administrator%40vsphere.local:
<pass>@vcenter.lab.local/"
```

Deploy OVA

```
ovftool --acceptAllEulas --skipManifestCheck --noSSLVerify \
  --diskMode=thin --powerOn \
  --name=<vm-name> \
  --datastore=vcenter-cl01-ds-vsan01 \
  --net:"Network 1"="vcenter-cl01-vds01-pg-vm-mgmt" \
  --prop:vamitimezone=UTC \
  --prop:root_password='<pass>' \
  "/path/to/appliance.ova" \
  "vi://administrator%40vsphere.local:<url-encoded-
pass>@vcenter.lab.local/vcenter-dc01/host/vcenter-cl01/"
```

Note: *Use single-line commands. Backslash continuation breaks* `--noSSLVerify` .

---

26. Windows Host — DNS & Firewall

DNS Record Management (PowerShell)

```
# Forward record
Add-DnsServerResourceRecordA -Name "vcenter" -ZoneName "lab.local" -IPv4Address
"192.168.1.69"

# Reverse record
Add-DnsServerResourceRecordPtr -Name "69" -ZoneName "1.168.192.in-addr.arpa" -
PtrDomainName "vcenter.lab.local"

# Verify
nslookup vcenter.lab.local
nslookup 192.168.1.69

# List all records
Get-DnsServerResourceRecord -ZoneName "lab.local"
```

Windows Certificate Utility

```
certutil -dump cert.crt
certutil -verify cert.crt
certutil -hashfile file.zip SHA256
```

---

27. Diagnostic Python Scripts

Run from Windows workstation. Requires `pip install paramiko`.

Scenario	Script
Overall health dashboard	`python quick_status.py`
NSX recovering after boot	`python nsx_monitor.py`
NSX VIP + node connectivity	`python nsx_check.py`
NSX performance + services	`python nsx_diag.py`
SDDC Manager vs NSX sync	`python sddc_nsx_status.py`
Diagnose remediate failure	`python check_remediate_error.py`
Clear stale DB locks + fix NSX status	`python clear_locks.py`
Fix stuck IN_PROGRESS tasks	`python fix_stuck_tasks.py`
Full cascade fix (all-in-one)	`python full_remediate_fix.py`
Check disconnected accounts	`python check_disconnected.py`
Update NSX password	`python nsx_cred_update.py`
NSX CPU overloaded	`python nsx_slim.py`
Restore NSX services	`python nsx_restart_all.py`
System clean after fix?	`python final_check.py`

> *All scripts are in `C:\VCF-Depot\diagnostic-scripts\`. Requires `pip install paramiko`. See `Diagnostic-Scripts-Cheatsheet.md` for full reference.*

---

28. GRUB Recovery — SDDC Manager

When all accounts are locked (root, vcf, admin):

1. Reboot VM in VMware Workstation
2. Press `e` at GRUB menu
3. Find the `linux` line, append: `init=/bin/bash`
4. Press `Ctrl+X` to boot
5. At bash prompt:

```
mount -o remount,rw /
faillock --user root --reset
faillock --user vcf --reset
reboot -f
```

> ***Note:*** *`systemctl` commands will not work in `init=/bin/bash` mode (systemd not running). Just reset faillock and reboot.*

Document Information

Field	Value
Document Title	VCF 9.0.1 Command Reference
Version	1.1
Last Updated	February 2026
Environment	Dell Precision 7920, VMware Workstation Nested Lab
Lab IPs	SDDC Mgr .241, vCenter .69, NSX VIP .70, NSX Node .71, ESXi .74-.76/.82